

REST API



Наставник: Казьмин Роман

Дата: 21.05.2026



Кому будет полезно?

Junior-разработчикам

Понять, что такое REST и почему он везде.
Научиться проектировать правильные URL,
использовать HTTP-методы по назначению и
читать чужой API без боли.

Frontend-разработчикам

Разобраться, почему CORS блокирует запросы, как
работает авторизация через токены и почему
fetch() вернул 403 вместо данных.

Студентам и стажёрам

Получить структурированную базу по REST API
перед первым проектом или техническим интервью
— чтобы отвечать уверенно, а не наугад.

QA-инженерам

Понять структуру API-запросов, научиться читать
статус-коды и тестировать эндпоинты через
Postman осознанно, а не методом тыка.



План вебинара

0

Вступление

1

Зачем вообще API?

2

Ресурсы и URL

3

HTTP методы и статус-коды

4

Запросы и ответы

5

Безопасность

6

Версионирование

7

Итог и Q&A



Блок 0

Вступление

5 минут

REST API — язык, на котором разговаривают все современные сервисы

Джун сталкивается с ним в первую же неделю на любом проекте

Большинство проблем в интеграциях — это нарушение базовых принципов REST

После этого вебинара у вас будет чёткая картина: что правильно, что нет и почему



Блок 1

Зачем вообще API?

10 минут

Блок 1 — Зачем вообще API?

Боль: хаос без стандартов

Как разным клиентам работать с одними данными? Прямой доступ к БД, бинарный RPC, XML... Всё это — путь к боли.

В нулевых доминировал SOAP — XML поверх HTTP с жёсткими схемами. REST появился как ответ на эту сложность.

API как контракт:

Сервер говорит: «Если ты пришлёшь мне вот это — я отвечу вот этим». Клиент не знает, что внутри сервера.

REST — скорее соглашение, чем спецификация. Нет жёсткого стандарта. Поэтому встречаются «REST API», которые нарушают базовые принципы.

REST — это не библиотека и не протокол

Набор архитектурных ограничений (Филдинг, 2000)

Работает поверх обычного HTTP

Читаемые URL

Легко тестировать — curl или Postman

Огромная экосистема инструментов



6 ограничений REST по Филдингу

1 Client-Server

Разделение интерфейса и логики

Клиент не знает про БД. Сервер не знает про UI. Общение только через API — можно развивать независимо.

2 Stateless

Никаких сессий в памяти сервера

Сервер не помнит контекст. Каждый запрос самодостаточен: ID, токен и данные — всё внутри.

3 Cacheable

Экономия трафика и ресурсов

Ответ помечается: можно кэшировать или нет (Cache-Control). CDN / прокси отдадут кэш, не трогая бэкенд.

4 Uniform Interface

Главный закон предсказуемости

Ресурсы по URL (/users/42), методы HTTP (GET/POST...), JSON + статус-коды. Один стандарт для всего.

5 Layered System

Клиент не знает, с кем говорит

За одним URL может быть Nginx, балансировщик, API Gateway, микросервисы — клиенту всё равно.

6 Code on Demand

Опциональное правило

Сервер может вернуть исполняемый код (JS-скрипт). В современном REST используется крайне редко.



Блок 2

Ресурсы и URL

15 минут

Блок 2 — Ресурсы и URL: существительное, не глагол

Плохо — глагольные URL:

```
POST /getUser
POST /createNewUser
POST /deleteUserById?id=42
POST /getUserOrders
```

Правильно — ресурсы:

```
GET    /users
POST   /users
GET    /users/42
DELETE /users/42
```

Правила именования:

- Существительные во множественном числе: /users, /orders, /products
- Нижний регистр, через дефис: /user-profiles (не /userProfiles)
- Никаких глаголов в URL: не /getUser, не /createOrder

Вложенность:

```
GET /users/42/orders — заказы
пользователя
GET /users/42/orders/7 — конкретный
заказ
```

Не уходи глубже 2 уровней вложенности —
становится больно читать



Блок 2 — Path vs Query параметры

Path параметр — конкретный ресурс по ID

GET /users/42 — конкретный пользователь

Query параметр — фильтрация, сортировка, пагинация

GET /users?role=admin&active=true — фильтрация
GET /products?page=2&limit=20 — пагинация

Примеры:

GET /users/42

GET /users?id=42

GET /users?role=admin

GET /products?page=2&limit=20



Блок 3

HTTP методы и статус-коды

15 минут

Блок 3 — HTTP методы

Метод	Действие	Тело запроса?	Идемпотентный?
GET	Получить данные	Нет	Да
POST	Создать ресурс	Да	Нет
PUT	Заменить целиком	Да	Да
PATCH	Изменить часть	Да	Зависит
DELETE	Удалить ресурс	Нет	Да

Идемпотентность — что это?

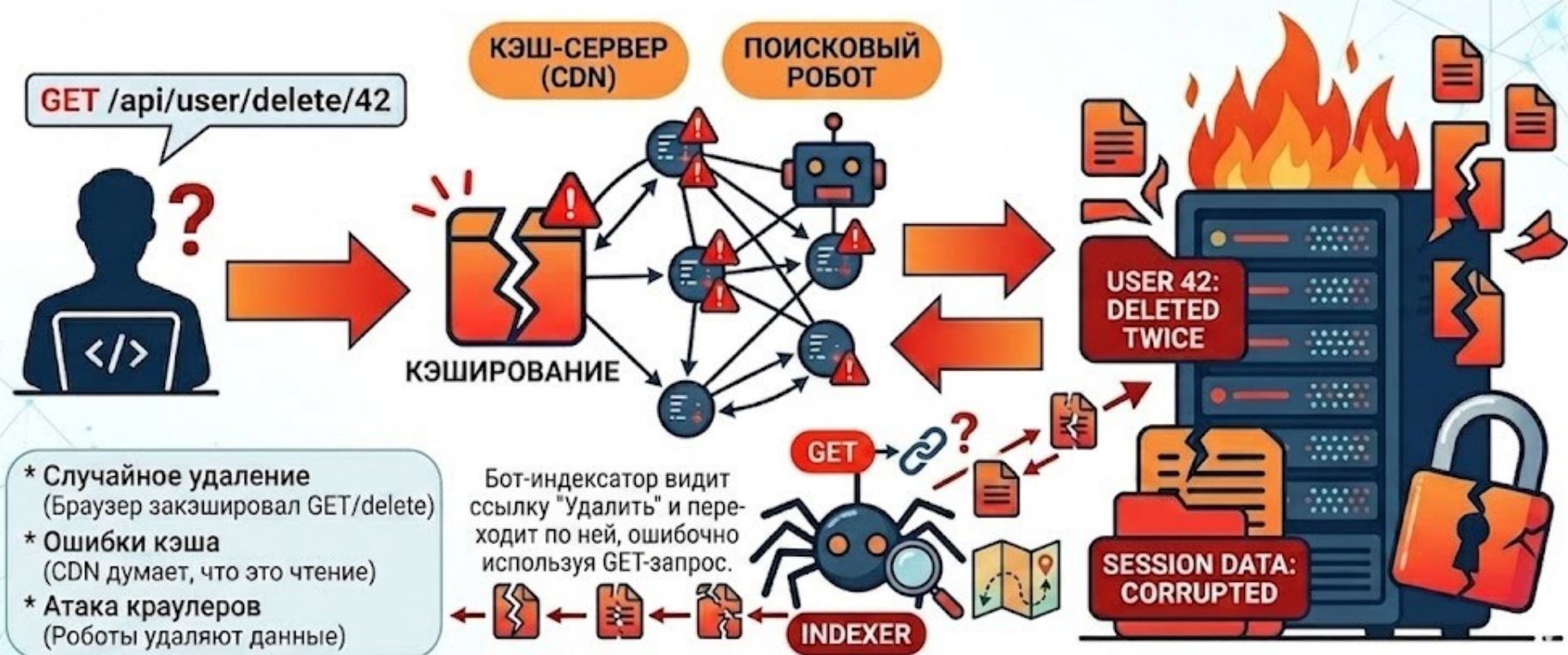
Идемпотентный запрос можно повторить любое количество раз с одним и тем же результатом.

DELETE /users/42 дважды: второй раз вернёт 404, но система не сломается. Это идемпотентно.

POST /orders дважды: создаст два заказа.
Не идемпотентно!



ПРОБЛЕМА: НАРУШЕНИЕ СЕМАНТИКИ HTTP (ИДЕМПОТЕНТНОСТЬ)



ПРОБЛЕМА: СЕТЕВАЯ АВТОНОМИЯ И retries (ПОВТОРЫ)



Блок 3 — статус коды

200 OK

Успешный запрос

201 Created

Ресурс создан (после POST)

204 No Content

Успешно, тела нет (DELETE)

400 Bad Request

Невалидные данные от клиента

401 Unauthorized

Нет или неверный токен — представься

403 Forbidden

Токен есть, но прав не хватает

404 Not Found

Ресурс не найден

422 Unprocessable

Структура ок, но данные не прошли валидацию

500 Internal Error

Ошибка сервера — клиент не виноват

503 Unavailable

Сервер временно недоступен

Главная ошибка: возвращать 200 OK с { "error": "not found" } в теле. Клиент думает, что всё хорошо — и начинает обрабатывать ответ как успешный.



Блок 4

Проектирование запросов и ответов

10 минут

Блок 4 — Запросы и ответы

Ключевые заголовки:

Content-Type: `application/json`

Формат тела запроса. Обязателен при POST/PUT/PATCH

Accept: `application/json`

Какой формат ответа ждёт клиент

Authorization: `Bearer <token>`

Токен авторизации (подробнее в блоке Безопасность)

Структура ответа — ошибка:

```
{ "error": "validation_error",  
  "message": "Email уже используется",  
  "details": { "field": "email" } }
```

Пагинация:

```
GET /users?page=2&limit=20  
  
{ "data": [...],  
  "total": 150, "page": 2,  
  "limit": 20, "pages": 8 }
```

Консистентность важнее красоты. В одном эндпоинте `user_id`, в другом `userId` — это боль для всех. Выбери одно соглашение и держись его.



Блок 5

Безопасность

15 минут

Блок 5 — CORS

CORS — почему браузер блокирует запросы

По умолчанию браузер не даёт JS на example.com делать запросы к api.other.com.

Почему? Чтобы вредоносный сайт не мог от твоего имени делать запросы к твоему банку.

Preflight-запрос (для сложных запросов):

```
OPTIONS /users HTTP/1.1
Origin: https://myapp.com
Access-Control-Request-Method: POST

← Access-Control-Allow-Origin: https://myapp.com
← Access-Control-Allow-Methods: GET, POST, DELETE
```

Важно:

CORS — это браузерная защита.

curl и Postman его не проверяют.

Access-Control-Allow-Origin: *
Нельзя использовать вместе с
Authorization — браузер заблокирует!

CORS не защищает API — он защищает пользователей браузера. Серверная авторизация всё равно нужна.



Блок 5 — Аутентификация: API Key и JWT

API Key — простой способ

```
GET /data HTTP/1.1
X-API-Key: sk_live_abc123xyz
```

Плюсы: просто. Хорошо для server-to-server.
Минусы: статичный — утёк, нужно перевыпускать.

Bearer Token / JWT

```
GET /profile HTTP/1.1
Authorization: Bearer eyJhbGci...
```

JWT = header.payload.signature (base64)
Payload содержит id, роль, срок жизни токена.
Сервер проверяет подпись — без запроса в БД.

Что внутри JWT payload:

```
{  "sub": "42",           // id пользователя
   "role": "admin",      // роль
   "exp": 1735689600     // срок жизни (unix timestamp)
}
```

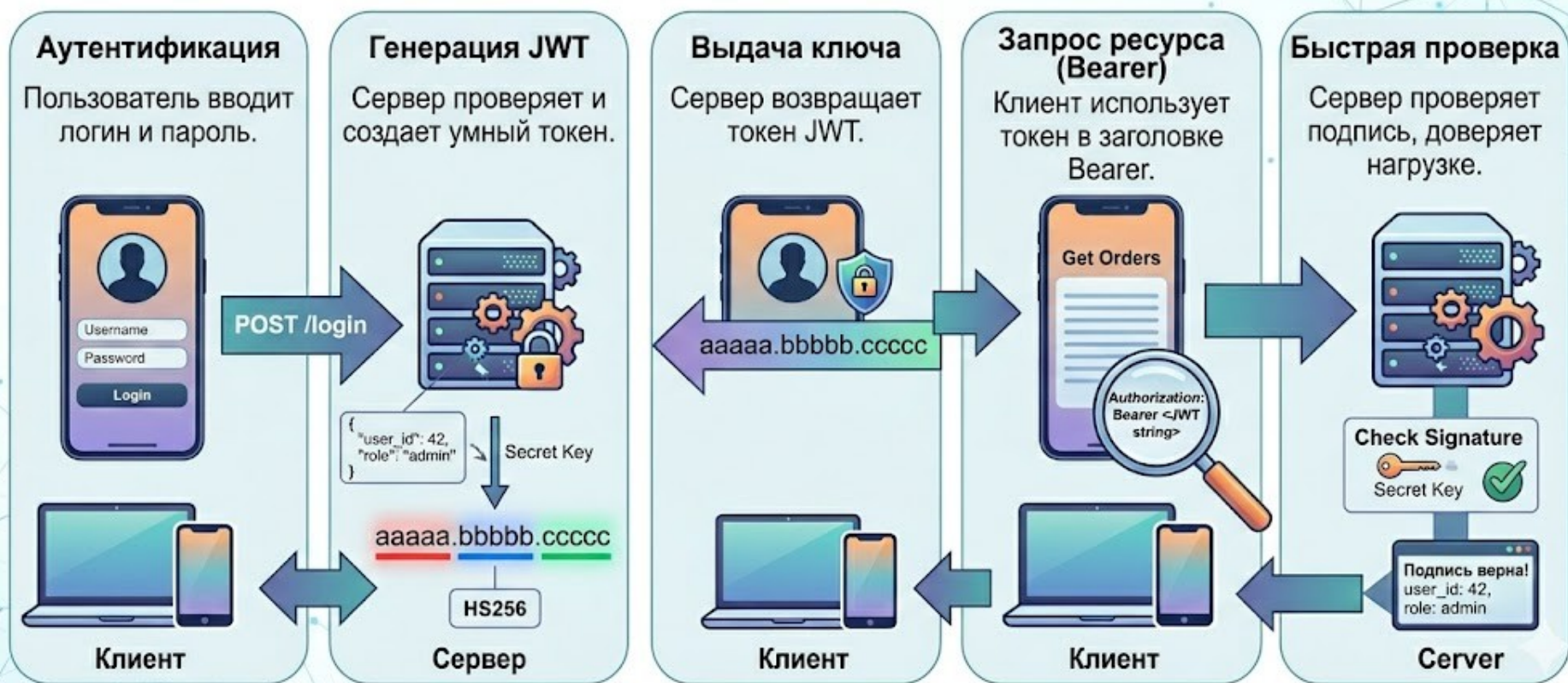
JWT payload не зашифрован — только подписан. Любой может прочитать содержимое.
Не кладите туда пароли и секреты.

OAuth 2.0 — следующий уровень:
делегирование авторизации (войти через Google). Это отдельная большая тема.



Сценарий взаимодействия: Bearer Token и JWT

Часть 3. Как они работают вместе (Пошаговый сценарий)



Блок 6

Версионирование API

5 минут

Блок 6 — Версионирование

Зачем? Ты поменял структуру ответа — сломал мобильное приложение на телефонах пользователей.

API — это публичный контракт. Нельзя просто взять и изменить.

★ Рекомендуется

1. Версия в URL

```
GET /v1/users
GET /v2/users
```

Очевидно, легко тестировать
— все так делают

Пуристы REST против, но на
практике это норма

2. Версия в заголовке

```
GET /users HTTP/1.1
API-Version: 2
```

URL чистый

Неочевидно, сложнее
тестировать в браузере

3. Content Negotiation

```
Accept:
application/
vnd.api.v2+json
```

«Правильно» по HTTP

Громоздко, плохо
поддерживается инструментами



Выпускаешь v2 — не удаляй v1 сразу. Дай клиентам время мигрировать. Добавь заголовок
Deprecation: true для предупреждения.

Блок 7

Итог и Q&A

5 минут

Блок 7 — Чеклист хорошего REST API

URL описывает ресурс (существительное),
не действие

CORS настроен корректно (не * при
использовании Authorization)

HTTP метод несёт семантику: GET читает,
POST создаёт, DELETE удаляет

Авторизация есть — хотя бы API Key или
Bearer Token

Статус-коды точно отражают результат (не
200 OK для ошибок)

Версионирование есть — хотя бы /v1/ в
URL

Консистентная структура запросов и
ответов по всему API

Пагинация есть — не возвращаешь все
записи сразу



Спасибо за внимание

Заполняйте анкету после активности!
Вам несложно,
Хекслету - полезно.

